

JUnit 单元测试进阶特性实验报告要求

一、实验概述

1. 实验名称

使用JUnit及Hamcrest构造测试用例

2. 实验目的

1. 掌握**JUnit** 参数化测试的核心用法，实现多组测试数据的统一测试，减少冗余代码。
2. 理解 测试套件（**Test Suite**）的作用，实现多个测试类的批量执行与管理。
3. 熟练使用**Hamcrest** 匹配器编写可读性更高、表达更灵活的断言语句。
4. 整合三种技术完成完整的单元测试场景，提升单元测试的规范性、复用性和可维护性。
5. 培养规范化编写测试用例、分析测试结果的能力。

3. 实验环境

- JDK 8 及以上
- 构建工具：Maven / Gradle（推荐 Maven）
- 测试框架：JUnit 4
- 断言库：Hamcrest
- IDE：IntelliJ IDEA / Eclipse

二、实验知识点

1. JUnit 参数化测试

- 适用场景：同一方法，多组输入输出数据的批量测试。

2. 测试套件

- 作用：将多个独立的测试类 / 测试方法打包，统一执行、统一管理。

3. Hamcrest 断言

- 替代原生 `assertEquals`，使用自然语言风格的匹配器。
- 常用匹配器：`equalTo`、`not`、`containsString`、`hasSize`、`greaterThan`、`allOf`、`anyOf` 等。
- 核心语法：`assertThat(实际值, 匹配器)`。

三、实验内容与要求

（一）基础业务代码实现

编写至少**2** 个业务类作为测试目标（必须包含可测试的逻辑方法，禁止无意义空方法，最好测试上学期的期末实训项目）。

要求：业务逻辑清晰，方法具备输入参数和返回值，适合单元测试验证。

（二）JUnit 参数化测试实现

针对上述业务类编写参数化测试类。

（三）测试套件实现

1. 创建测试套件类，将本次实验的所有测试类（参数化测试类、普通测试类）加入套件。
2. 要求：
 - 能够一键执行套件内所有测试用例。
 - 套件配置规范，注解使用正确。
 - 测试套件执行后，能展示所有用例的执行结果（成功 / 失败 / 跳过）。

（四）Hamcrest 灵活断言表达式实现

1. 全面替换 JUnit 原生断言（`assertEquals`/`assertTrue`/`assertFalse`）。
2. 必须使用以下

Hamcrest 核心匹配器

- 基础匹配：`equalTo`、`not`

- 字符串匹配: `containsString`、`blankOrNullString`
- 数字匹配: `greaterThan`、`lessThan`
- 组合匹配: `allOf`（同时满足）、`anyOf`（任一满足）
- 集合 / 对象匹配: `hasSize`、`nullValue`

3. 要求: 断言语句语义清晰, 符合自然语言阅读习惯。

（五）实验结果与分析

1. 截图:

- 测试套件执行结果截图。
- 参数化测试多组用例执行截图。
- Hamcrest 断言代码及测试通过截图。

2. 分析:

- 统计测试用例总数、成功数、失败数。
- 说明参数化测试相比普通测试的优势。
- 对比 Hamcrest 与原生断言的区别, 说明灵活表达式的价值。
- 分析失败用例原因（若有）并给出修复方案。

四、实验总结要求

1. 总结本次实验掌握的核心技能。
2. 说明参数化测试、测试套件、**Hamcrest**在实际项目中的应用价值。
3. 记录实验过程中遇到的问题及解决方案。
4. 谈谈对单元测试规范化的理解。

总结

这份实验报告要求完整覆盖了 **JUnit** 参数化测试、测试套件、**Hamcrest** 断言 三大核心知识点, 明确了实验目的、内容、代码规范、格式要求和评分标准, 可直接用于课程实验、作业提交或学习实践。

你可以直接基于该要求编写实验代码与报告, 若需要配套示例代码（业务类 + 测试类 + 套件类）, 我可以为你补充完整。